



University of Sri Jayewardenepura
Faculty of Computer Science

**ICT 1051-Database Systems and
Administration**

Sales Management System
Java based GUI application with MySQL Database

Index No: AS20240953

Name: Chanuki Semini Hettiarachchi

1. Introduction

This report presents the database design component of the Sales Management System developed for the ICT 1062 Object-Oriented Programming module. The system is a Java-based desktop application that simulates a retail business environment where administrators can manage products, customers, and sales transactions.

The database backend is built using MySQL, a popular open-source relational database management system. The database is designed following normalization principles up to Third Normal Form (3NF) to ensure data integrity, eliminate redundancy and maintain consistency across all tables.

This report covers the complete database design including the Entity Relationship Diagram, table definitions, relationships, normalization analysis, constraints, indexes, views and stored procedures.

2. System Overview

The Sales Management System requires a database to store the following types of information:

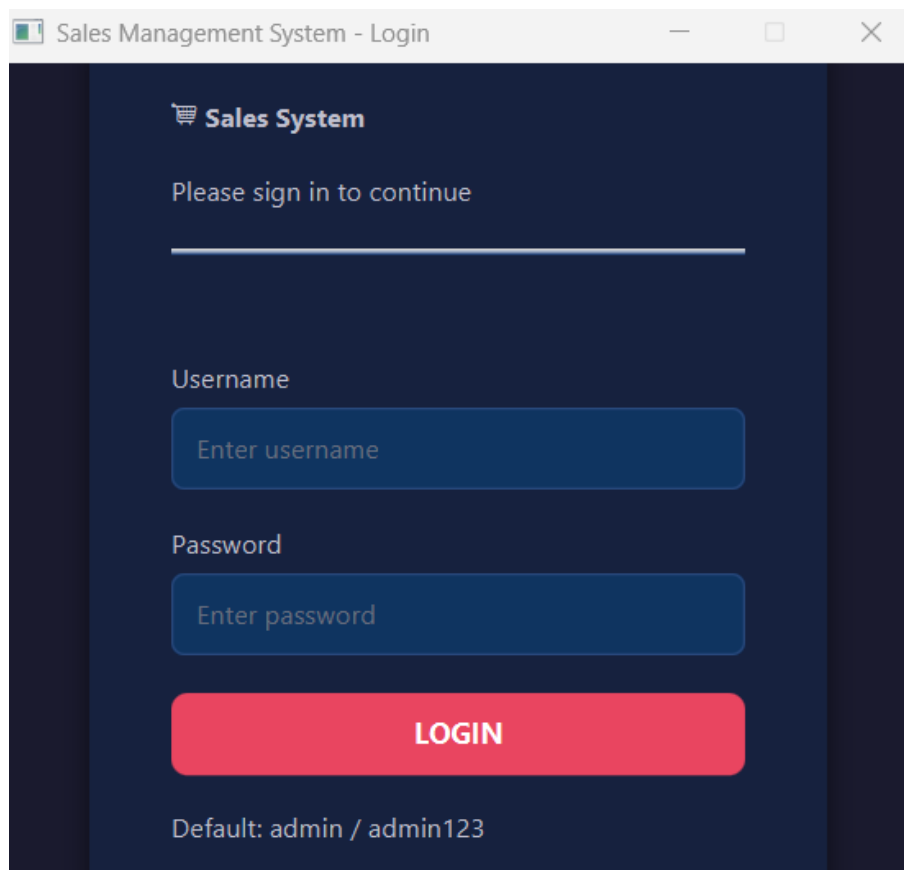
- User accounts for system login and authentication
- Product catalog with names, prices, and stock quantities
- Customer records with contact information
- Sales transaction records with dates and total amounts
- Individual line items within each sale transaction

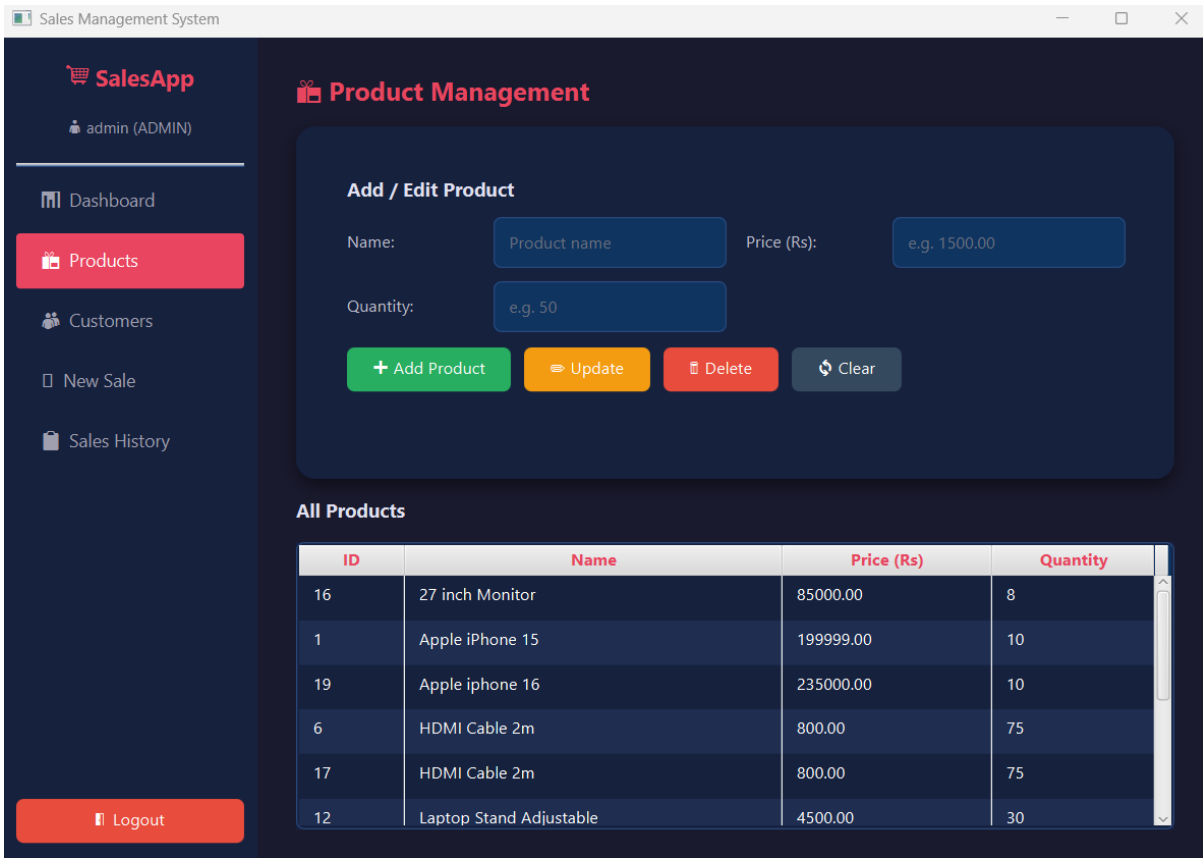
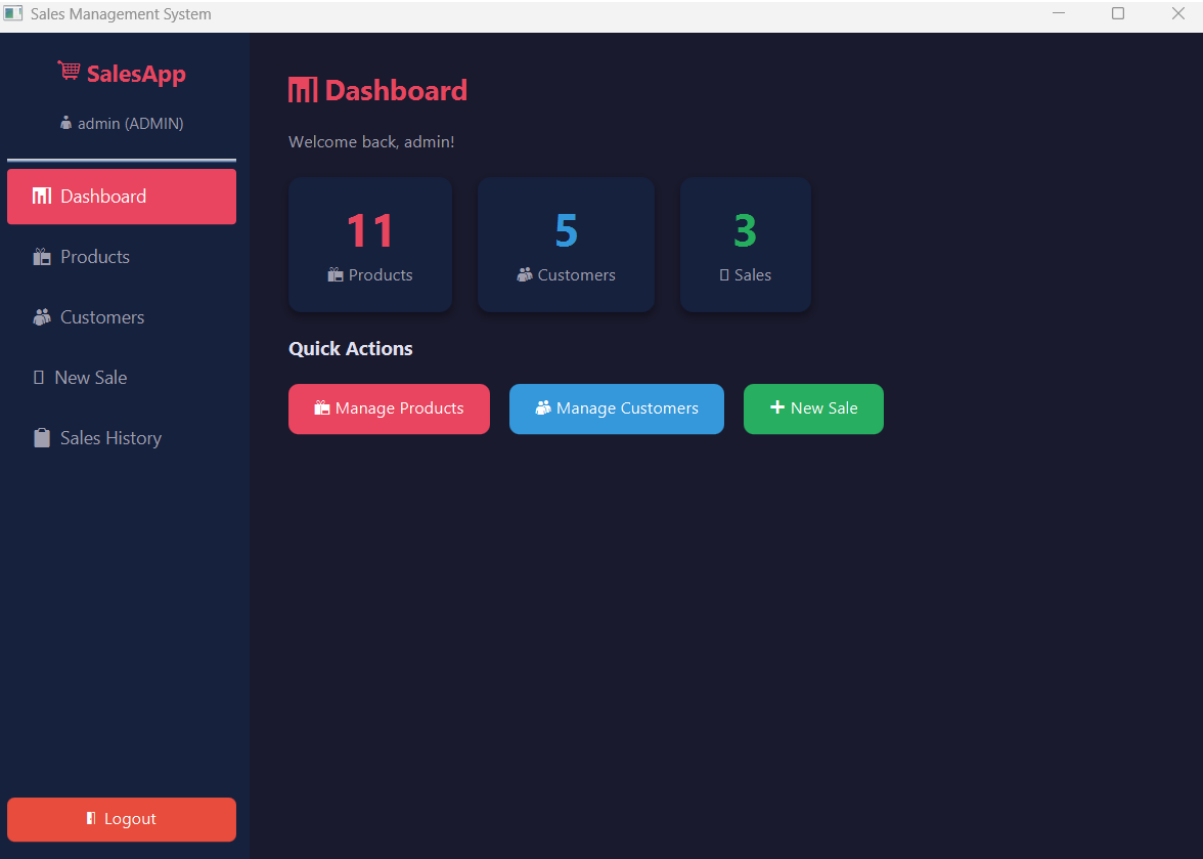
The database consists of five interrelated tables that together support all the functional requirements of the system. The tables are designed to minimize data redundancy while maintaining referential integrity through the use of foreign key constraints.

3. GUI Screenshots

The system consists of six main screens as described below. Screenshots should be taken from the running application and inserted here.

Screen	Description
Login Screen	Username and password fields with a LOGIN button. Shows error messages for invalid credentials.
Dashboard	Home screen showing counts of products, customers, and sales with quick action buttons.
Product Management	to add/edit/delete products with a table showing all products.
Customer Management	Form to add/edit/delete customers with a table showing all customers.
New Sale	Select customer and products, add to cart, apply discounts and complete the sale.
Sales History	View all past sales and click 'View Invoice' to see a popup receipt.
Invoice Popup	A formatted receipt showing customer details, items, and total amount.





Sales Management System

SalesApp

admin (ADMIN)

Dashboard

Products

Customers

New Sale

Sales History

Logout

Customer Management

Add / Edit Customer

Name:

Customer full name

Contact:

e.g. 0771234567

+ Add Customer

Update

Delete

Clear

All Customers

ID	Name	Contact
9	chanuki semini	0776892059
10	Diwantha Radishan	0779876541
2	Nimali Silva	0769876543
3	Ruwan Fernando	0712345678
4	Sanduni Wickrama	0754321098

Sales Management System

SalesApp

admin (ADMIN)

Dashboard

Products

Customers

New Sale

Sales History

Logout

New Sale Transaction

Select Customer & Product

Customer:

Select customer

Product:

Select product

Unit Price:

Auto-filled

Quantity:

Enter qty

In stock: -

+ Add to Cart

Order Summary

Subtotal:

Rs 0.00

Discount %:

0

Discount Rs:

Rs 0.00

TOTAL:

Rs 0.00

Complete Sale

Clear Cart

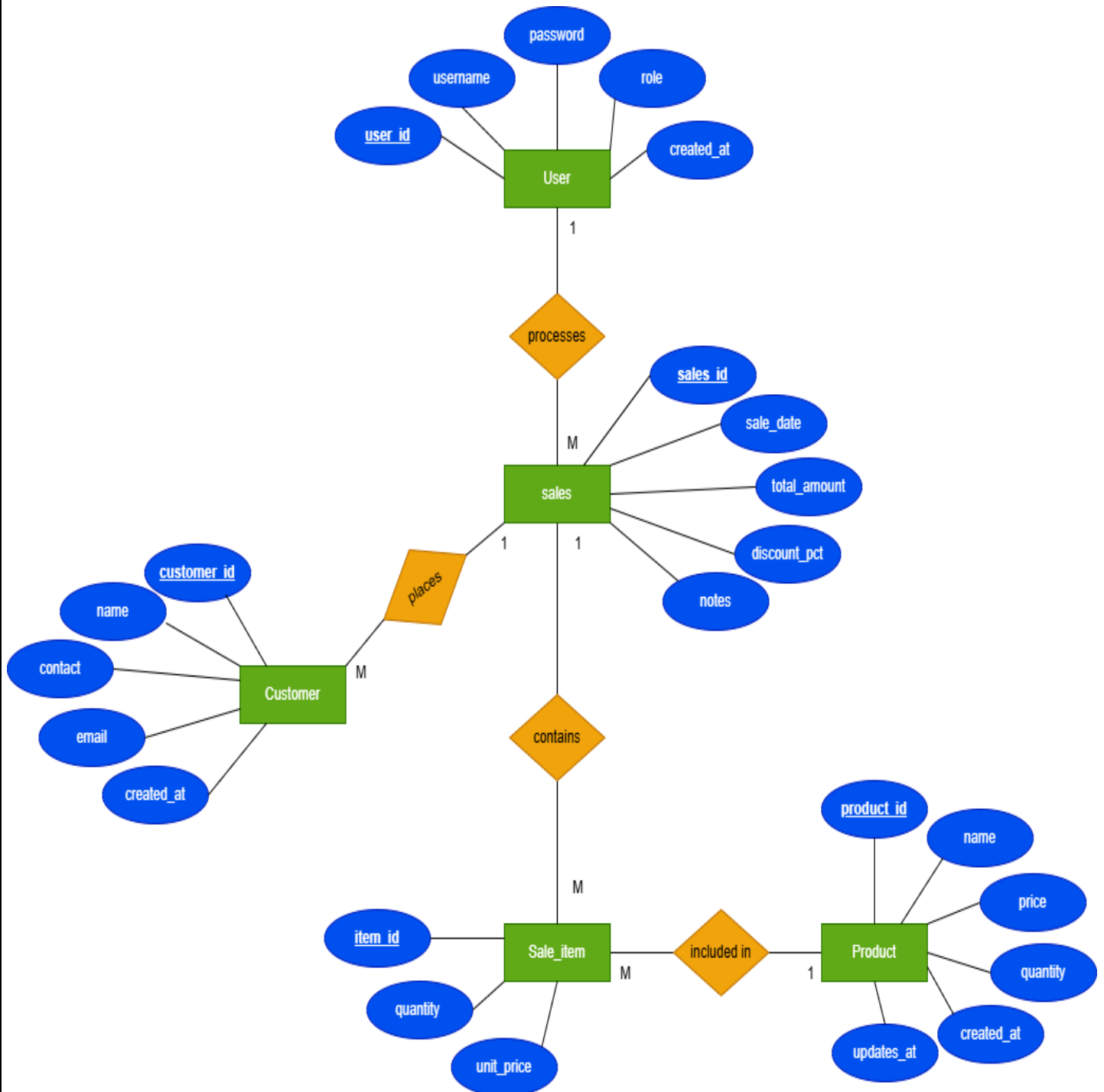
Cart Items

Product	Qty	Unit Price	Subtotal	Remove
Cart is empty. Add products above.				

4. Database Design

4.1 Entity Relationship Diagram

The ERD below illustrates the five entities in the system and the relationships between them.



4.1.1 Entities and Relationships

Entity	Description	Relationship
users	Stores system login accounts	Independent. so, no FK relationships
products	Stores the product catalog	One product appears in MANY sale_items
customers	Stores customer information	One customer places MANY sales
sales	Stores each sale transaction	One sale contains MANY sale_items
sale_items	Stores products within a sale	Links sales and products together

4.1.2 Relationship Types

Relationship	Type	Explanation
customers to sales	One-to-Many (1:M)	One customer can place many sales, but each sale belongs to only one customer
sales to sale_items	One-to-Many (1:M)	One sale can contain many items, but each item belongs to only one sale
products to sale_items	One-to-Many (1:M)	One product can appear in many sale items across different sales
users	Independent	Users only manage login access and have no relationship with other entities

4.1.3 Table Definitions

Table 1: users

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(50)	NOT NULL, UNIQUE	Login username
password	VARCHAR(255)	NOT NULL	Login password
role	ENUM	NOT NULL, DEFAULT USER	ADMIN or USER role
created_at	DATETIME	NOT NULL	Account creation date

Table 2: products

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique product identifier
name	VARCHAR(150)	NOT NULL	Product name
price	DECIMAL(12,2)	NOT NULL, CHECK >= 0	Product selling price
quantity	INT	NOT NULL, CHECK >= 0	Available stock quantity
updated_at	DATETIME	AUTO UPDATE	Last modification date

Table 3: customers

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique customer identifier
name	VARCHAR(150)	NOT NULL	Customer full name
contact	VARCHAR(20)	NOT NULL, UNIQUE	Contact phone number
email	VARCHAR(100)	NULL	Optional email address
created_at	DATETIME	NOT NULL	Record creation date

Table 4: sales

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique sale identifier
customer_id	INT	FK -> customers.id	Links sale to a customer
sale_date	DATETIME	NOT NULL	Date and time of sale
total_amount	DECIMAL(12,2)	NOT NULL, CHECK >= 0	Final total after discount
discount_pct	DECIMAL(5,2)	DEFAULT 0.00	Discount percentage applied

Foreign Key Behavior:

- ON DELETE RESTRICT - Cannot delete a customer who has existing sales
- ON UPDATE CASCADE - If customer ID changes, automatically updates in sales table

Table 5: sale_items

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique item identifier
sale_id	INT	FK -> sales.id CASCADE	Links item to a sale
product_id	INT	FK -> products.id	Links item to a product
quantity	INT	NOT NULL, CHECK > 0	Quantity sold
unit_price	DECIMAL(12,2)	NOT NULL	Price at time of sale

Important Design Decision: The unit_price is stored in sale_items rather than being read from the products table. This is because product prices may change in the future, but the invoice must always show the price that was charged at the time of the sale.

Foreign Key Behavior:

- sale_id ON DELETE CASCADE - If a sale is deleted, all its items are deleted too
- product_id ON DELETE RESTRICT - Cannot delete a product that has been sold

4.1.4 Normalization

All five tables in the database have been normalized to Third Normal Form (3NF). Below is the analysis for each normalization level.

First Normal Form (1NF)

A table is in 1NF if:

- All columns contain atomic (indivisible) values
- Each column contains only one type of data
- Each row is unique

How our database satisfies 1NF:

- Every column stores a single value. For example, name stores only the name, price stores only the price
- No column stores multiple values or lists
- Every table has a PRIMARY KEY that uniquely identifies each row
- No repeating groups exist. Sale items are stored in a separate sale_items table, not as multiple columns in the sales table

Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and all non-key attributes are fully dependent on the entire primary key (no partial dependencies).

How our database satisfies 2NF:

- All tables use a single column primary key (id), so partial dependency cannot exist
- In sale_items. Ex: quantity and unit_price depend on BOTH sale_id AND product_id together. This is why we have a composite unique constraint on (sale_id, product_id)
- Customer name and contact depend only on the customer id, not on any sale information

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and there are no transitive dependencies (non-key columns depending on other non-key columns).

How our database satisfies 3NF:

- In the sales table, total_amount depends on sale_id, not on customer_id. Customer details are stored separately on the customers table
- In sale_items, unit_price is stored at the time of sale and does not depend on the current price in the products table. This eliminates a transitive dependency
- No column in any table depends on another non-key column

Table	1NF	2NF	3NF	Reason
users	YES	YES	YES	All attributes depend only on id. No transitive dependencies.
products	YES	YES	YES	Price and quantity depend only on product id.
customers	YES	YES	YES	contact and email depend only on customer id, not on name.
sales	YES	YES	YES	total_amount stored for history and does not depend on customer data.
sale_items	YES	YES	YES	unit_price stored at time of sale, independent of products table.

4.1.5 Constraints and Data Integrity

Every table has a primary key constraint on the id column. This ensures every row is uniquely identifiable and prevents duplicate records.

Foreign Key Constraints

Table	Foreign Key	References	On Delete	On Update
sales	customer_id	customers(id)	RESTRICT	CASCADE
sale_items	sale_id	sales(id)	CASCADE	CASCADE
sale_items	product_id	products(id)	RESTRICT	CASCADE

CHECK Constraints

CHECK constraints enforce business rules at the database level:

Table	Constraint Name	Rule	Purpose
products	chk_products_price	price >= 0	Prevents negative product prices
products	chk_products_quantity	quantity >= 0	Prevents negative stock quantities
customers	chk_customers_contact	LENGTH(contact) >= 7	Ensures valid contact numbers
sales	chk_sales_total	total_amount >= 0	Prevents negative sale totals
sales	chk_sales_discount	discount_pct BETWEEN 0 AND 100	Ensures valid discount percentage

sale_items	chk_sale_items_quantity	quantity > 0	Ensures at least 1 unit is sold
------------	-------------------------	--------------	---------------------------------

UNIQUE Constraints

- users.username - No two users can have the same username
- customers.contact - No two customers can share the same phone number
- sale_items(sale_id, product_id) - The same product cannot appear twice in one sale

4.1.6 Indexes

Indexes are created on columns that are frequently used in WHERE clauses and JOIN operations to improve query performance.

Index Name	Table	Column	Purpose
idx_products_name	products	name	Speed up product name searches
idx_customers_name	customers	name	Speed up customer name lookups
idx_sales_customer_id	sales	customer_id	Speed up JOIN with customers table
idx_sales_sale_date	sales	sale_date	Speed up date-based filtering
idx_sale_items_sale_id	sale_items	sale_id	Speed up JOIN with sales table
idx_sale_items_product_id	sale_items	product_id	Speed up JOIN with products table

4.1.7 Views

Three views are created to simplify complex queries and provide useful data summaries.

v invoice details

Purpose: Joins all four main tables to provide complete invoice information in a single query.

Columns returned: sale_id, sale_date, discount_pct, total_amount, customer_name, customer_contact, product_name, quantity, unit_price, line_total

v customer sales summary

Purpose: Shows a summary of how many sales each customer has made and their total spending.

Columns returned: customer_id, customer_name, contact, total_sales, total_spent

v low stock

Purpose: Shows all products where the quantity is below 10 units, ordered by lowest stock first. Used for stock alerts.

Columns returned: id, name, price, quantity

4.1.7 Stored Procedure

sp_get_sale_invoice

Purpose: Retrieves all details for a specific sale including customer information and all purchased items. This procedure is used to generate the invoice receipt in the application.

Input Parameter: p_sale_id (INT) - The ID of the sale to retrieve

Output: A result set containing sale_id, sale_date, total_amount, discount_pct, customer_name, contact, product_name, quantity, unit_price and subtotal for each line item.

Example Usage:

```
CALL sp_get_sale_invoice(1);
```

5. Conclusion

The database design for the Sales Management System demonstrates a well-structured relational database that follows industry best practices. The design achieves the following:

- Full normalization to Third Normal Form (3NF) eliminating data redundancy and updating anomalies
- Referential integrity enforced through properly configured foreign key constraints with appropriate ON DELETE and ON UPDATE behaviors
- Business rule enforcement through CHECK constraints that prevent invalid data such as negative prices or quantities
- Query performance optimization through strategic use of indexes on frequently joined columns
- Code reusability through views that simplify complex multi-table queries
- Advanced database functionality demonstrated through a stored procedure for invoice retrieval
- Historical data accuracy maintained by storing unit_price in sale_items rather than referencing the products table

The five table structure with clear relationships between customers, sales, sale_items, products and users provides a solid and scalable foundation for the Sales Management System. All design decisions have been made with data integrity, performance and maintainability in mind.

References

- MySQL AB. (2024). MySQL 8.4 Reference Manual. Retrieved from <https://dev.mysql.com/doc/>
- Codd, E. F. (1970). A relational model of data for large, shared data banks. Communications of the ACM, 13(6), 377-387.
-
- Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.
- Oracle Corporation. (2024). Java SE 17 API Documentation. Retrieved from <https://docs.oracle.com/en/java/javase/17/>